Jack Heinrich and Steven Chang

Team Leader: Jack Heinrich

Professor Yin

CMPSC 448

10/11/2026

Midterm Project 1 Report

Problem Definition

Large Language Models (LLMs) have become widely used across a variety of tasks, including writing and generating text. As a consequence, identifying which LLM generated a piece of text has become important. Attributing AI-generated text to an LLM is critical for maintaining accountability across models. Although LLMs can adopt different tones and writing styles, there still remain some underlying characteristic differences across various models' text. In this project, we trained a CNN and an RNN (LSTM) so that they, given an LLM response and/or the corresponding prompt, output one of five labels (ChatGPT, Claude, Mistral AI, Copilot, or Qwen) predicting which model produced the response. We evaluated our models' accuracy on a test set against a 20% chance accuracy baseline. Using our models, we answered four research questions:

- **RQ1:** Can we identify which LLM generated a response from the response alone?
- **RQ2:** Does the user's prompt help identify the LLM? (input only vs. output only vs. both)
- **RQ3:** Do LLM fingerprints generalize across tasks or domains? (train on one domain, test on another)
- **RQ4:** What characteristics distinguish different LLMs?

Dataset Curation

The source of the dataset is TXD-22 (Iqbal & Abul Kashem, 2026b; also on Kaggle), a public dataset accompanying a peer-reviewed paper (Iqbal & Abul Kashem, 2026a) under the CC BY 4.0 license. We excluded the data file from our repository as it was too large to track on GitHub. Replication of our results requires downloading the dataset manually from the cited page in our README. The size of the dataset is 66,000 rows (one question, answer and source per row) from 22 sources (17 AI tools and 5 human or mixed human/AI categories). Each source answers the same 3,000 questions in the same order: 100 questions for each of the 30 subjects. We opted to use ChatGPT, Claude, Mistral AI, Copilot, and Qwen for our sources. Other models like Llama and Gemini had more errors in their data, which would have reduced the total amount of usable data across all models, since we dropped a question for every model whenever any model's answer to it was flawed, to keep the classes balanced.

We modified the dataset as follows. In our data_pipeline.py file, we read the file using Latin-1, instead of UTF-8, since the CSV file mixes Windows-1252 and UTF-8 encoding. Then, we repaired the resulting garbled characters with the ftfy library. We also realigned Claude's rows, which were out of order relative to the other sources. We replaced every source's prompt wording with ChatGPT's, so that all models have identical prompts; otherwise input-only classification (RQ2) could learn from wording differences. Finally, we removed 143 questions whose answers were either cut off (5) or answered a different question (138), for all five models. After fixing the dataset, we had 2,857 questions remaining for each model. Since TXD-22 has no subject labels, we labeled each question's subject by its row position, identifying each 100-question block's subject by reading its questions, and grouped the 30 subjects into three domains (STEM, Social Science, and Humanities) for RQ3. We also split the dataset into a ratio

of 70% training, 10% validation, and 20% test data, which resulted in 1,997, 291, and 569 questions per source respectively (9,985, 1,455, and 2,845 rows in total). The split is by question, so all models' answers to a question land in the same set, and it is done within each subject with a fixed random seed.

Neural Net Implementations

Both of our neural network implementations share the same text pipeline and training infrastructure. Our text pipeline is adapted from Tran (2020), with changes for our task described below. The text pipeline prepares the text data for training and testing by tokenizing, building a vocabulary, encoding, and packing the data into tensor datasets. Our regex tokenizer replaces Tran's NLTK tokenizer and tokenizes by splitting a piece of text into a list of the lowercased individual words that make it up; punctuation marks are treated as their own tokens. When using both prompts and answers for classifying, we concatenate each prompt to its response with a <sep> token separating them. The pipeline then builds a vocabulary, a mapping of all tokens that are seen at least twice to a number, from the training split only (answers, questions, or both, depending on the RQ2 condition). Unlike Tran, who builds it from all the data, this keeps information from the validation and test sets out of training. Numbers 0 and 1 are reserved for two special tokens, <pad> (for padding short texts to a common length) and <unk> (tokens not in the vocabulary), so the remaining tokens are numbered from 2 on. Tokens that are seen fewer than twice are replaced with <unk> because they could teach the models to only associate that one token with whatever source it came from, and mapping them to <unk> also teaches the models how to handle unseen words at test time. Every text is cut or padded to a fixed length of 512 tokens for answers (the median answer is 306 tokens and the 99th percentile is 528), 32 for questions, and 528 for questions and answers together. Tran pads every sentence to the longest

one, but our answers are far longer than his movie-review sentences (up to 1,490 tokens), so we truncate to a fixed length instead. The text pipeline then encodes all training, validation, and test rows from lists of tokens to lists of numbers using the vocabulary as a map. Finally the pipeline packs the encoded rows into tensor datasets and batches them into groups of 32, shuffling only the training data. These tensor datasets can then be used by our neural networks for training and testing.

Both models are trained and evaluated with the same shared training code, which works with any PyTorch model that takes token IDs and outputs one score per LLM. For both models, the loss function is cross-entropy loss. This is the negative log of the probability the model assigns to the correct label, so as the models train, the probability given to the correct answer increases. Both models use the Adam optimizer at its default learning rate of 0.001. Adam adapts each weight's learning rate using running averages of its past gradients. It is one of the most widely used optimizers in deep learning today, which is why we selected it. Both models train for 10 epochs; after each epoch the model is evaluated on the validation set and the weights from the epoch with the best validation accuracy are kept and used for the final evaluation on the test set. This guards against overfitting in later epochs. Lastly, both models use a manually set seed to make our results reproducible, though GPU computation can still cause small differences between repeated runs. In RQ3, each model is trained once per domain, and that same trained model is tested on all three domains, so differences in those results come from the domain alone.

CNN

Our CNN follows Tran's (2020) PyTorch implementation of a CNN for sentence classification with two main differences. First, our CNN model's embeddings are randomly initialized and learned from scratch, whereas Tran's model uses pretrained fastText vectors.

Second, Tran uses Adadelta as his optimizer whereas we chose to use Adam. The hyperparameters of our CNN were kept from Tran's implementation (300-dimensional embeddings, 100 filters per window size, dropout of 0.5) without tuning.

Following Tran's implementation, our model uses four stages, with a few operations in between. First, the data goes through the embedding layer. This layer transforms each token ID into a learned 300-number vector. The padding token's vector stays all zeros, because it is never updated during training. Second are the three parallel 1D convolution layers. The three layers have windows of 3, 4, and 5 tokens and 100 filters each. They use ReLU as their activation function, and each filter looks at that many tokens at a time and learns to detect a particular phrase pattern. After the convolution layers output their scores, max pooling keeps each filter's strongest score across the whole text, so each feature records whether, and how strongly, a pattern appeared, regardless of its position. After getting each filter's strongest score, we concatenate them together from all three convolution layers to produce a 300-feature vector for each text. After concatenating the max pool results, we feed the results into the third layer which is the dropout layer. The dropout layer randomly sets half of these 300 features to zero during each training step, so the model cannot rely too heavily on any single filter. This reduces overfitting. Dropout is turned off when evaluating, so all features are used for validation and testing. Finally the results go into the fourth and final layer which is a linear layer that turns the 300 features into 5 scores, one per LLM; the highest score is the model's prediction. A CNN is well suited to answering our research questions because each filter acts as a feature detector, picking up on certain phrases characteristic of particular models. Despite our CNN's capabilities, it can only see windows of up to 5 tokens, meaning it can't capture long stretches of context, and

may miss certain patterns. An RNN such as an LSTM, which reads the text in order, can capture longer context; ours is described next.

RNN

Unlike the CNN, which looks at the short windows of the text independently, an RNN reads one token at a time and updates the carried hidden state:

$$h_t = tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

We used a long short-term memory (LSTM) network (Hochreiter & Schmidhuber, 1997) rather than a plain RNN since our answers are long with 306 median tokens. A plain RNN would rewrite its entire hidden state at every token (as seen in the equation), so by the last iteration the beginning tokens would have little to be remembered. More importantly, during training the error signal backpropagates through every token, and in a plain RNN it would become very small (ex. $0.9^{306} \approx 10^{-14}$) meaning the model would have little to learn, which is the vanishing gradient problem (Bengio et al., 1994). An LSTM solves this with a separate cell state memory that is updated by addition and controlled by learned forget, input and output gates; when the forget gate stays open, information and the error signal pass through many steps almost unchanged.

Our RNN uses PyTorch's built-in LSTM layer and has four stages, like the CNN. First, an embedding layer turns each token ID into a learned 300-number vector, as in the CNN. Second, the batch is truncated before reaching the LSTM. Most of our answers are within the length of 512 tokens, so this design choice only influences 1.2% of our data where the lengthier answers are cut to fit the matrix. We also count each answer's real length from its non-padding tokens and use PyTorch's pack_padded_sequence so that the

LSTM stops at each answer's last real token. Then, a single bidirectional LSTM layer with 128 hidden units per direction processes each direction (front to end, and end to front). An LSTM's final state is mostly influenced by the end, so the bidirectional layer accounts for both front and back fairly. Finally, identical to the CNN, a dropout layer ($p = 0.5$) and a linear layer turn the 256 features into 5 scores, one per LLM. The embedding size and dropout rate match the CNN design, so the two models can be compared fairly. The model has about 8 million parameters, most of it consisting of the embedding layer.

Results

All results below are accuracies on the test set of 2,845 answers (569 per LLM), using each model's weights from its best validation epoch in a single training run. With five balanced classes, random guessing would achieve 20% accuracy.

Table 1: Test accuracy by model and condition (chance = 20%)

Model	Input only	Output only	Both
CNN	20.0%	77.0%	75.7%
LSTM	20.0%	72.0%	72.7%

Figure 1: CNN confusion matrix, output only (test results)

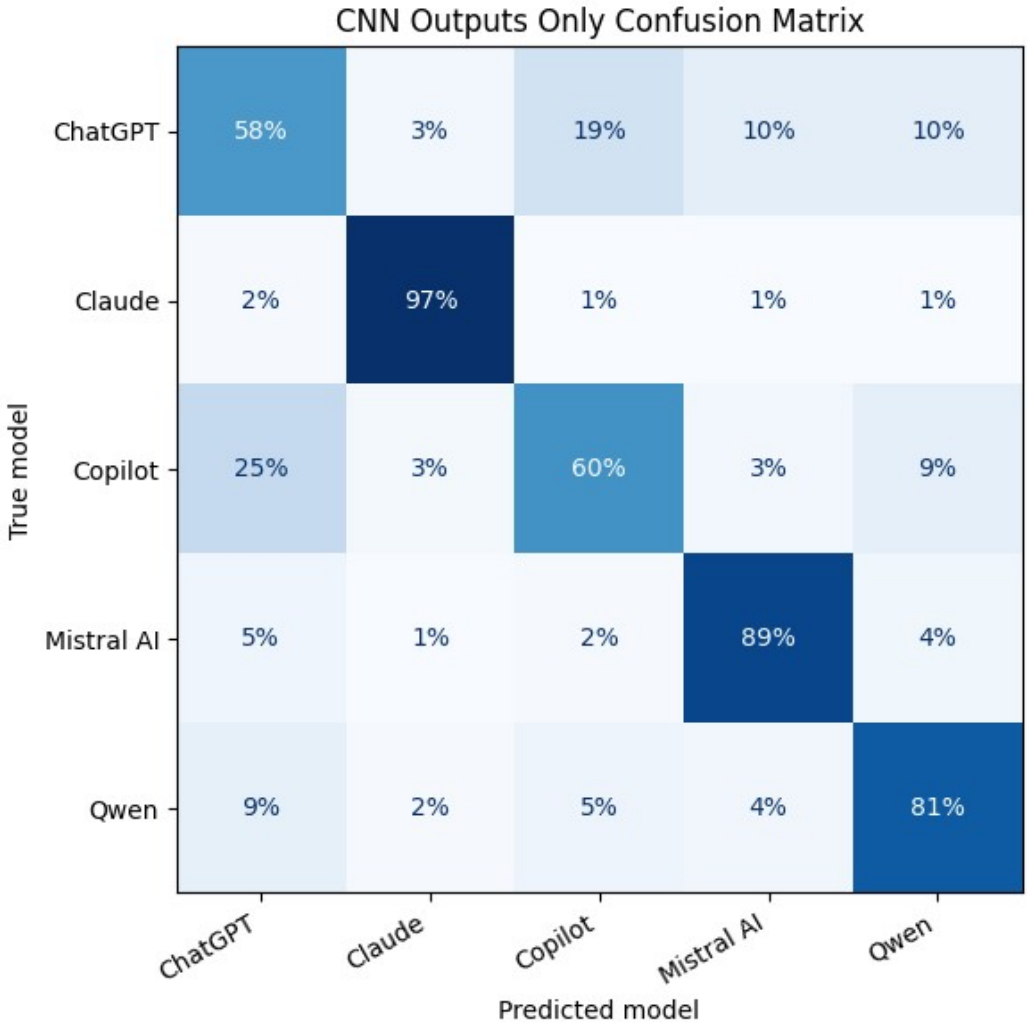


Figure 2: LSTM confusion matrix, output only (test results)

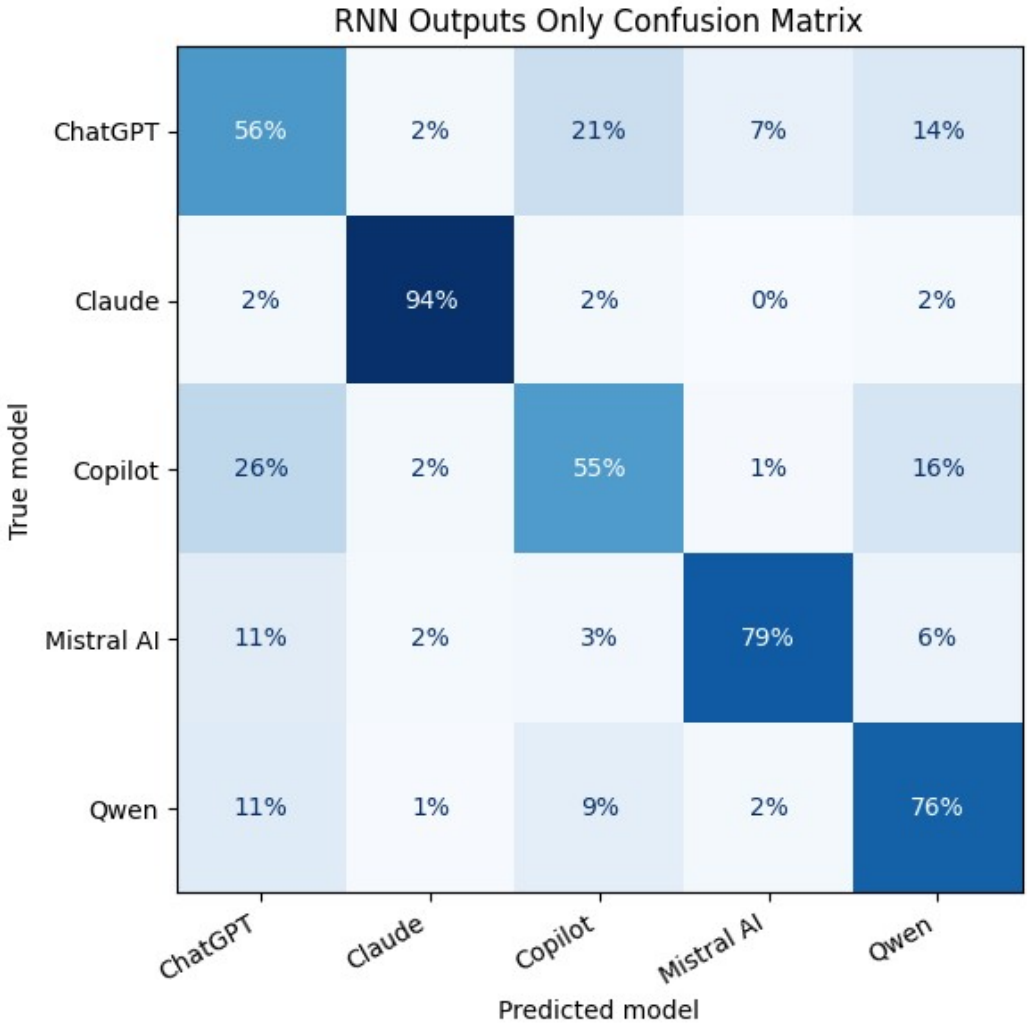


Figure 3: CNN training curves

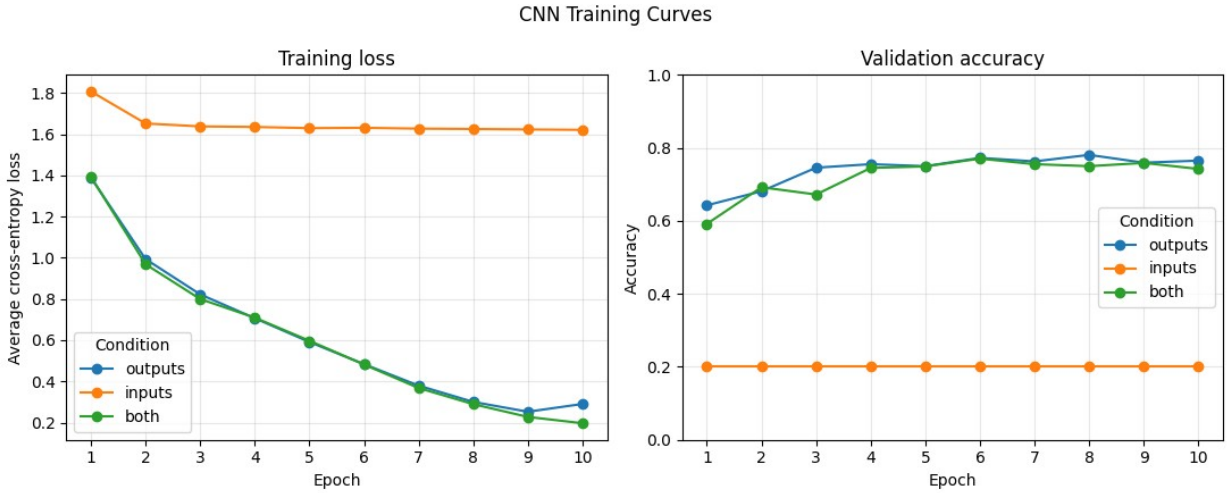


Figure 4: LSTM training curves

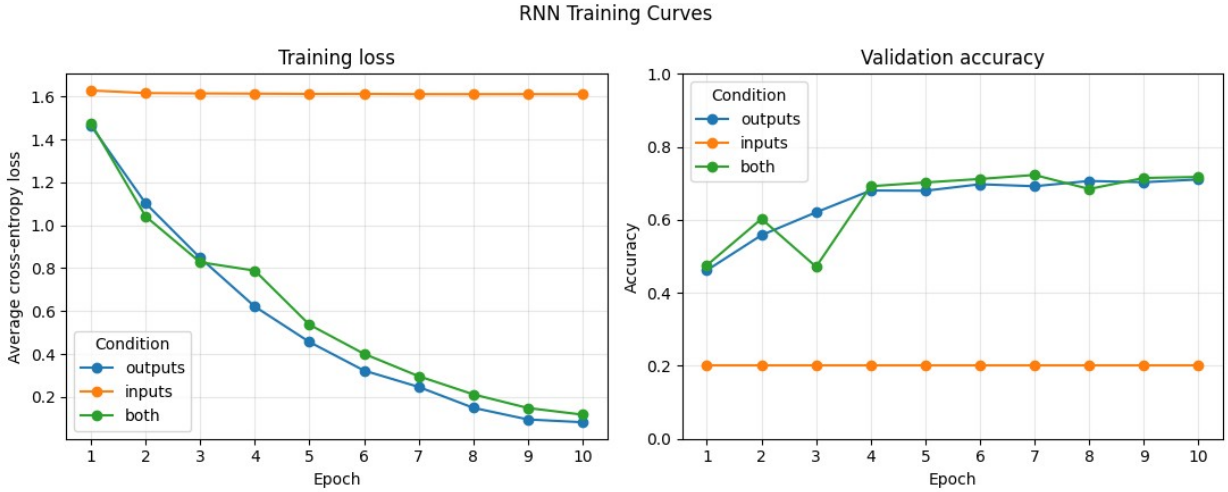


Figure 5: CNN cross-domain accuracy

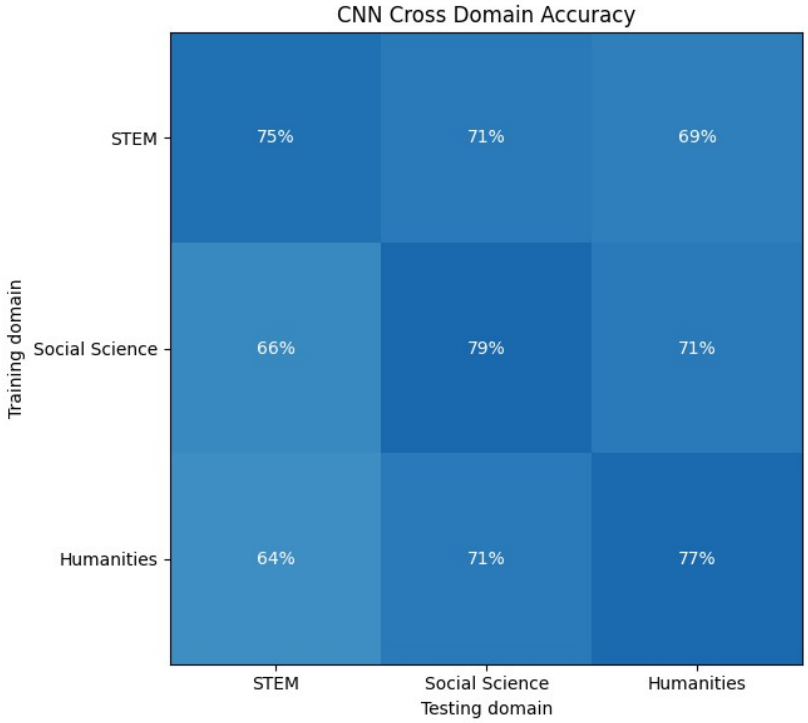


Figure 6: LSTM cross-domain accuracy

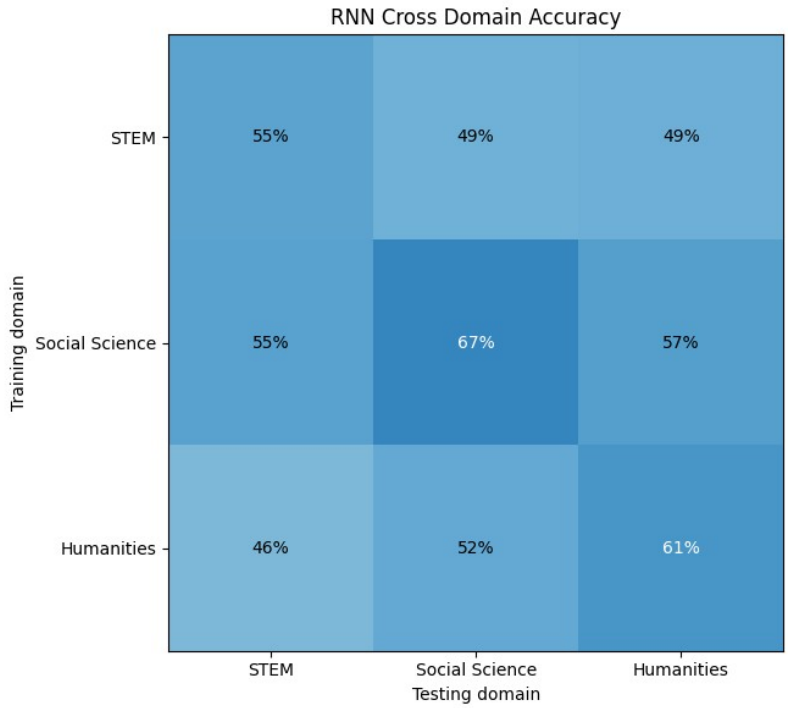


Table 2: Domain accuracy comparison

Domain \ Tested on	STEM	Social Science	Humanities	Average
CNN in-domain	75.3%	78.5%	77.4%	77.1%
CNN cross-domain (avg of 2)	64.8%	71.3%	70.3%	68.8%
LSTM in-domain	54.5%	67.0%	60.5%	60.7%
LSTM cross-domain (avg of 2)	50.5%	50.4%	53.0%	51.3%

RQ1

The first question we set out to answer was "Can we identify which LLM generated a response from the response alone?" Our results show that the answer is yes. After training our CNN and LSTM on LLM outputs alone, our models achieved 77.0% and 72.0% accuracy, respectively, at identifying which LLM generated a given response (Table 1). These accuracies far exceed the 20% expected from random guessing, showing that LLM responses contain consistent patterns that neural networks can learn. As shown in our confusion matrices (Figures 1 & 2), Claude was the easiest to identify (97% CNN, 94% LSTM), followed by Mistral AI and Qwen, while ChatGPT and Copilot were the hardest, with both models only accurately identifying them around 55 to 60% of the time, mostly because they were confused with each other.

RQ2

The second question was "Does the user's prompt help identify the LLM?" However, this time the answer was no. When trained using only the prompts given to the LLMs, both models scored exactly 20.0% (Table 1). All five LLMs received the exact same prompts, meaning each prompt appears once for every model, so it carries no information about which LLM answered it.

Since the same input always gets the same prediction, exactly one of the five copies of each question can be classified correctly, giving exactly 20%. This is observable as we can see both models' loss quickly level off at about 1.61, which is -ln(0.2). This tells us the models are assigning every LLM an equal 20% probability which is the best they can do when the input has no identifiable patterns. Validation accuracy also stays at exactly 20% in every epoch (Figures 3 & 4).

Another angle we explored was whether training the model with the input concatenated with the output could improve classification. Predictably, adding the input made very little difference in the performance of both models since the prompt alone carries no information about the source. After adding the input, the CNN and LSTM achieved accuracies of 75.7% and 72.7% respectively (Table 1). The differences are small (1.3 points or less) and go in opposite directions, supporting that including the prompt does not meaningfully help identify the LLM.

RQ3

The third question was "Do LLM fingerprints generalize across tasks or domains?" As shown in Figures 5 and 6 as well as Table 2, we approached this problem by training the model with three different domains (STEM, Social Science, and Humanities), and tested in-domain accuracies as well as cross-domain accuracies. Comparing each domain's cross-domain accuracy with its in-domain accuracy separates the effect of the domain shift from differences in how hard each domain is. As the table suggests, in-domain tests had the highest accuracies of 77.1% for the CNN and 60.7% for the LSTM. On cross domains, we notice a moderate drop of accuracy with CNN dropping 8.3 percentage points and the LSTM dropping 9.4 percentage points. Accuracies stay far above the 20% chance level even across domains (the CNN's lowest cross-domain result is 63.8%), and for the CNN, STEM is the hardest domain to transfer to. One

thing to consider is that domain-specific training results in less training data which results in a noticeable drop in accuracy for the LSTM as LSTMs typically need more training data. This is supported by the fact that the LSTM trained on Social Science (the domain with the largest dataset) scores 55.4% on STEM, slightly higher than the LSTM trained on STEM itself (both appear as 0.55 in Figure 6). However, the models mostly learned a general fingerprint with some domain-specific patterns.

RQ4

To answer the fourth question, which is "What characteristics distinguish different LLMs?", we first created a Python script that shows which characteristics are noticeably different among the models. We measured different characteristics such as structures, formatting, punctuation, and more. Because the models answered the same set of questions, the frequencies of characteristics were the model's choice rather than topic variance. The following table shows the summary of meaningful discoveries we found:

Table 3: Linguistic characteristics by model

Characteristic	ChatGPT	Claude	Copilot	Mistral AI	Qwen
Words per answer	247	301	307	234	221
Words per sentence	24.2	41.1	26.0	28.5	23.1
% of answers containing a dash (— or –)	35%	5%	43%	3%	26%
Parentheses per 100 words	0.55	0.30	0.42	0.32	0.68
Average word length (letters)	5.87	6.40	6.05	5.74	6.26
Function-word share	0.37	0.32	0.35	0.41	0.32
Lexical diversity	0.84	0.86	0.85	0.77	0.87
Repeats an 8-word phrase	0.8%	0.7%	0.8%	17.8%	0.3%

Table 3 shows that the LLMs differ in both structural and linguistic ways. Claude writes by far the longest sentences (41.1 words, versus 23 to 28 for the others) and rarely uses dashes, while Mistral AI repeats an 8-word phrase in 17.8% of its answers and has the lowest lexical diversity. ChatGPT and Copilot have similar sentence lengths and both use dashes often, which matches how often our models confuse them. We then further investigated which signals the model actually used to classify the models by removing each signal and measuring the accuracy afterwards. We only tested it on CNN for simplicity, and the results are as follows:

Table 4: Accuracy of CNN after removing a signal

Signal removed (CNN only)	Accuracy	Drop
none	77.0%	-
Equal length (first 100 tokens)	59.9%	-17.1 points
Content words only (function words hidden)	66.0%	-10.9 points
Function words only (content words hidden)	72.8%	-4.2 points
No punctuation	73.5%	-3.5 points

The baseline of CNN's accuracy with no signals removed comes from the earlier test from RQ1 (Table 1). In each experiment, we removed length variance in answers, replaced function words with placeholder tokens, replaced content words with placeholder tokens, and removed all punctuation. At first glance, it might seem like length is the biggest factor to model differentiation. In our results, Copilot's recall falls from 60% to 25% after removing length differences, which fits it being the longest writer. However, cutting every answer to its first 100 tokens removes a lot of the answer, so there are fewer patterns to recognize from. Function words are a more reliable signal as hiding them costs 10.9 points, while hiding content words or removing punctuation costs only 4.2 and 3.5.

In-Depth Analysis and Experiments

The results from our models above reveal insights about the models we trained, as well as the LLMs that the data came from. Some of the results we can analyze to learn more about what they reveal are what caused most of our models' errors, how the CNN performed against the LSTM, and what the limitations of our experiments say about our results.

Error Analysis

While our models classified most LLMs well, they struggled disproportionately in telling two specific LLMs apart. The most common error for both models is confusing ChatGPT and Copilot in both directions. In the output-only condition, the LSTM labeled 26% of Copilot answers as ChatGPT, and 21% of ChatGPT answers as Copilot, and the CNN 25% and 19% (Figures 1 & 2). This was the most confusion between any two LLMs from the entire experiment. This is likely because Copilot is powered primarily by OpenAI's GPT models (Microsoft, 2025), so their responses could share distinguishing features. By contrast, Claude, whose style is the most distinctive, was identified correctly 94 to 97% of the time.

CNN vs LSTM

Based on our experiments, CNN achieved better accuracy compared to LSTM. CNN achieved higher recall for all five models with Claude being the easiest, and Copilot and ChatGPT being the hardest. Overall, they were able to find the same differences, but CNN found them more reliably. The difference comes from the architecture rather than model size. Both models have around 8 million parameters with about 95% of them being the embedding, so both have nearly the same size. RQ4 allowed us to discover why CNN suited this task better as well: CNNs have an easier time recognizing local patterns and word choices with their filters, while the LSTM must go through roughly 300 steps of hidden state updates for the final result.

Because neither model was tuned and each result comes from a single run, these conclusions apply to our configurations rather than the two architectures in general.

Limitations

Our results have two main limitations that affect how far our conclusions can be generalized. First, each result came from a single training run, and none of our models' hyperparameters were tuned to improve results. Because we only did one training run, small differences such as the gaps of 1.3 points or less in RQ2 could change if the models were retrained. While it may be possible to improve our models' accuracy through tuning the batch size, max length of our texts, or training epochs, we opted not to experiment with tuning as our results were clear enough to answer our research questions.

Second, every prompt in our dataset is a curated question that was identical across all five models. This was necessary for a fair comparison, as it prevented our classifiers from identifying models by the question topic rather than the LLMs' fingerprints. However, it does not reflect realistic LLM usage. For RQ2, our results show that the prompts carried no information about which LLM they were sent to, but in reality, people may send different types of prompts to different LLMs. For example, ChatGPT might receive more basic questions whereas Claude might receive more coding-related ones. Had our data been a collection of actual LLM interactions, our RQ2 results might be more generalizable. The dataset also stores responses as flattened text, without their original lists, bold text, or line breaks, which removed formatting as a feature our models could have used.

Lessons and Experience Learned

This project taught us lessons about what it takes to build and train a neural network that can succeed at its task. One of the most important lessons we learned was that a lot of work goes

into preventing overfitting. We ended up using more defenses against overfitting than we expected to need. Some examples are replacing tokens that occurred fewer than twice with <unk> tokens, using dropout layers, and keeping the weights from the epoch with the best validation accuracy rather than the last epoch. Even so, our training curves show signs of overfitting (Figures 3 & 4). The CNN's training loss kept decreasing, from 1.39 to under 0.3, while its validation accuracy leveled off between 75% and 78% after the fourth epoch. Its best validation accuracy came at epoch 8 and dropped afterward, so keeping the best epoch meant we tested better weights than the final ones. This taught us that no single technique prevents overfitting, and that validation accuracy, not training loss, shows whether a model is actually improving.

We also learned some fundamental differences between CNN and LSTM. While we expected the LSTM's ability to handle long-term dependencies to give it an advantage, the CNN performed better and trained about four times faster. This taught us that having a longer context window does not guarantee better results. In our experiment, the CNN's ability to capture local patterns mattered more, and the LSTM's context provided little benefit. We would expect the LSTM to do better if the responses kept their original structure and formatting, since those are long-range sequential patterns it is designed to learn.

References

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. https://doi.org/10.1109/72.279181

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9(8)*, 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Iqbal, M. S., & Abul Kashem, M. (2026a). SiBot: A hybrid framework for fine-grained semantic attribution of AI-generated text and its production deployment. *PLOS ONE, 21*(9), e0357405. https://doi.org/10.1371/journal.pone.0357405

Iqbal, M. S., & Abul Kashem, M. (2026b). *TXD-22: A large-scale benchmark dataset for multi-class AI-generated and mixed-source text identification* (Version 1) [Data set]. Mendeley Data. https://doi.org/10.17632/prcjcggtjf.1

Microsoft. (2025, August 7). *Microsoft incorporates OpenAI's GPT-5 into consumer, developer and enterprise offerings*. Microsoft Source. https://news.microsoft.com/source/features/ai/openai-gpt-5/

Tran, C. (2020, February 1). *A complete guide to CNN for sentence classification with PyTorch*. https://chriskhanhtran.github.io/posts/cnn-sentence-classification/